

QueryLearn: Visual System Guide, Architecture & Defense Handbook

Complete Engineering Diagrams, Screen-by-Screen Walkthrough, Cloud Architecture, and Oral Defense Answers

Project Overview: QueryLearn is an online research lab comparing whether student programmers learn data querying faster and with less frustration using declarative SQL or procedural Python.	Target Demographic: 2nd to 4th Year Computing Students (Sophomore, Junior, Senior in BSCS/BSIT/BSIS) with novice-to-intermediate backgrounds (1st-year freshmen excluded per panel guidelines).
Fair Testing Plan: A 2x2 Crossover Design where every student tries both languages in balanced Latin Square orders to eliminate order and learning bias.	Cloud Persistence: Shared Supabase PostgreSQL database ensures unique sequential Study IDs (P017, P018+) and live real-time telemetry across multiple laboratory computers.

The Core Comparison: Declarative SQL vs. Procedural Python

Feature	SQL (Declarative)	Python (Procedural / Loops)
How you think	You describe WHAT data you want. The database engine figures out how to fetch it.	You describe HOW to get the data, writing every single loop and dictionary step.
Simple filter	SELECT name FROM Students WHERE year = 2;	for s in students: if s['year'] == 2: result.append(s['name'])
Joining tables	JOIN Courses ON Enrollments.course_id = ...	Must build a dictionary lookup or write nested loops by hand.
What we study	Do novices find SQL natural, or do keywords feel like a foreign language?	Do novices prefer Python because they already know loops, or is it too tedious?

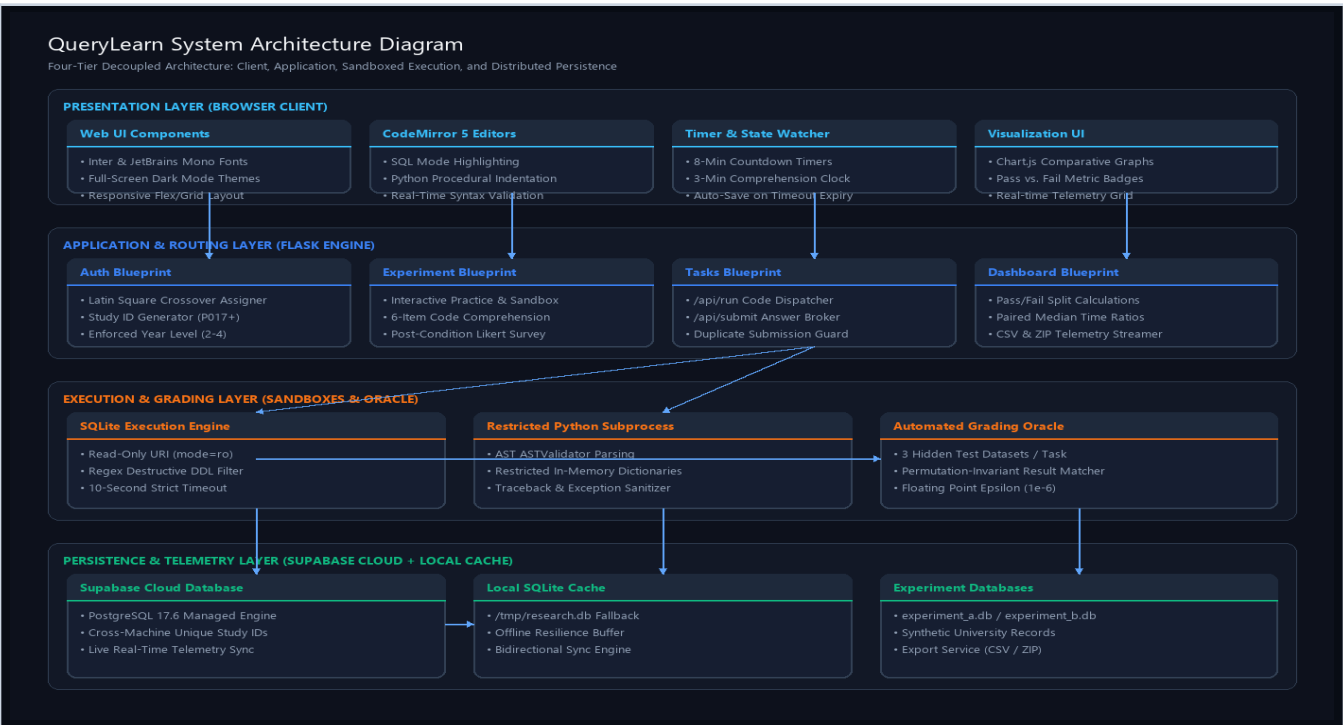
The 6 Relational Task Challenges (T1 to T6)

Task	Concept	What the Student Solves	SQL vs. Python Mapping
T1	Selection & Filtering	Find all 2nd-year BSIT students.	WHERE program='BSIT' AND year=2 vs. loop filter
T2	Sorting & Limiting	Get top 3 scores in course C102 (ignore empty scores).	IS NOT NULL, ORDER BY DESC, LIMIT 3 vs. [:3] slice
T3	Grouping & Counting	Count total students enrolled in each program.	GROUP BY program, COUNT(*) vs. dictionary counter
T4	Joining Tables	Show student name and course name together.	INNER JOIN across tables vs. dictionary index lookup
T5	Average per Group	Find courses with average score 85.0 or higher.	GROUP BY ... HAVING AVG() >= 85 vs. dict-of-lists mean
T6	Records With No Match	Find students who have zero enrollments.	LEFT JOIN ... WHERE id IS NULL vs. set difference

■ ■ **What you say to your teacher / defense panel:** "QueryLearn tests the core operations of relational databases: filtering, sorting, grouping, joining, aggregations, and anti-joins. Both languages solve the exact same tasks so the comparison is 100% fair."

1. System Architecture Diagram

Four-Tier Layered Architecture: Client Presentation, Application Routing, Sandboxed Execution, and Distributed Persistence

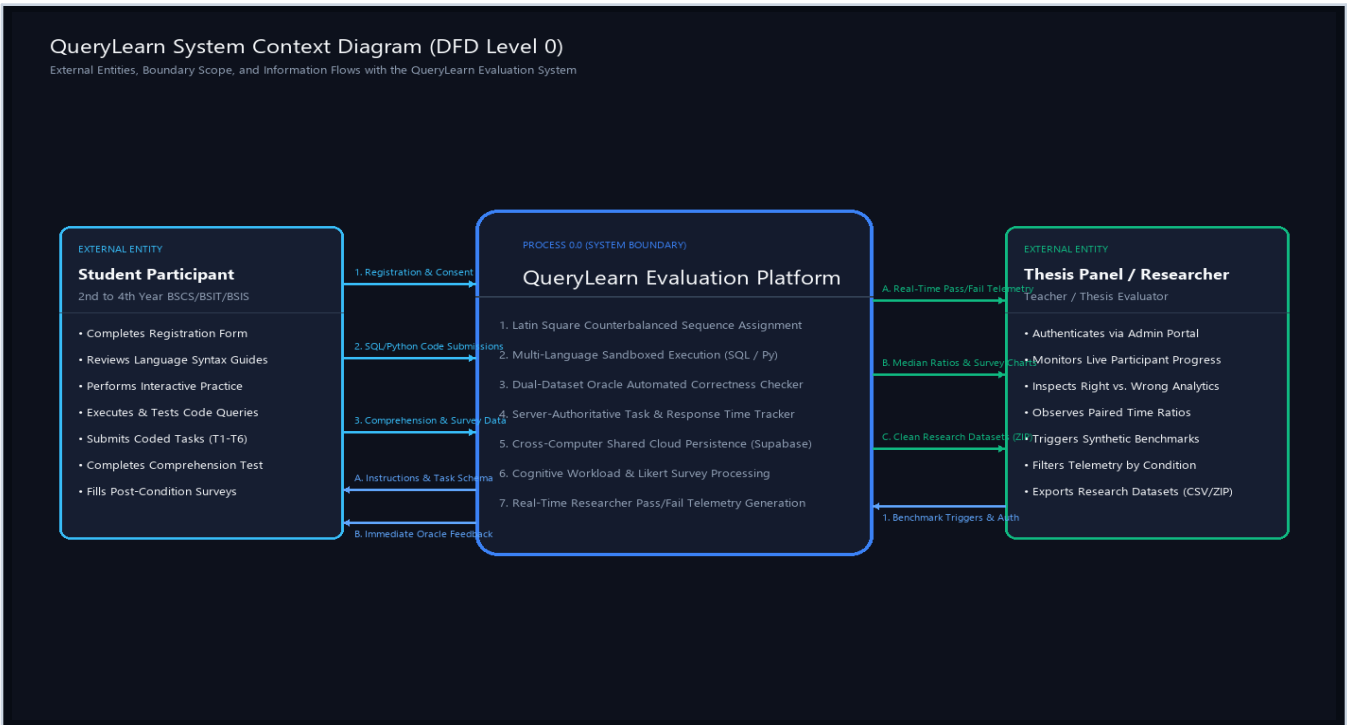


Formal Specification: Decoupled into four distinct layers: (1) Client Presentation Layer (CodeMirror 5 editors, server countdown clocks, Chart.js); (2) Server Application Layer (Flask Blueprints implementing Latin Square crossover sequencing); (3) Sandboxed Execution Layer (Read-Only SQLite URI engine mode=ro, restricted Python subprocess, and Automated Grading Oracle with 3 hidden test cases); and (4) Distributed Persistence Layer (Supabase PostgreSQL 17 for real-time cross-computer synchronization with local SQLite fallback).

■ ■ **What you say to your teacher / defense panel:** "We designed QueryLearn with a decoupled 4-tier architecture to guarantee security and multi-machine reliability. Student code is executed in isolated read-only sandboxes, while research telemetry synchronizes to a shared Supabase PostgreSQL cloud database so the evaluation panel can view live results across different laboratory computers simultaneously."

2. System Context Diagram (DFD Level 0)

High-Level Environmental Boundary, External Entities, and Bidirectional Information Flows

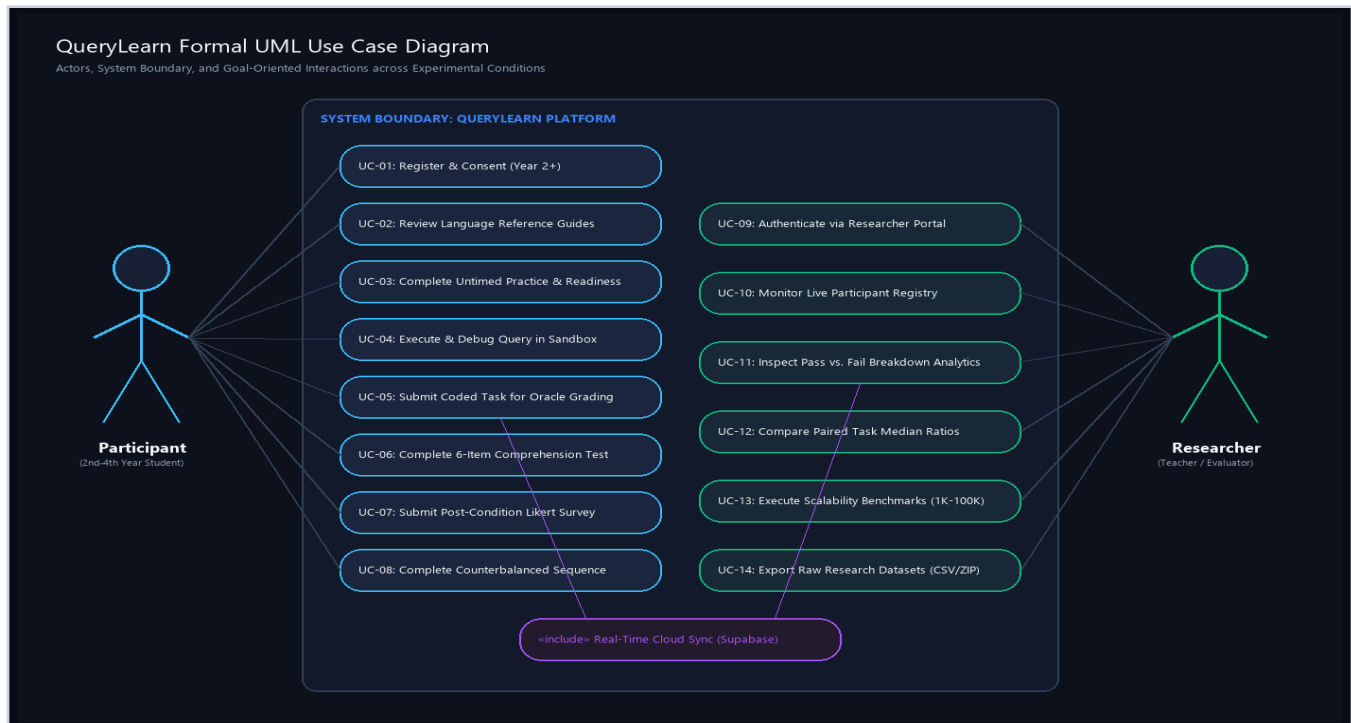


Formal Specification: The Level 0 Context Diagram formalizes the platform boundary and primary data flows between the system and its two external actors: the Student Participant (2nd to 4th year computing students providing registration demographics, code submissions, and Likert survey responses) and the Researcher / Thesis Panel (accessing real-time pass/fail telemetry, paired time ratios, and downloading CSV datasets).

■ **What you say to your teacher / defense panel:** "This context diagram proves the bounded scope of our research tool: QueryLearn is not an LMS or social platform, but a focused empirical evaluation instrument engineered strictly to measure and compare declarative SQL against procedural Python in a controlled, double-blind environment."

3. Formal UML Use Case Diagram

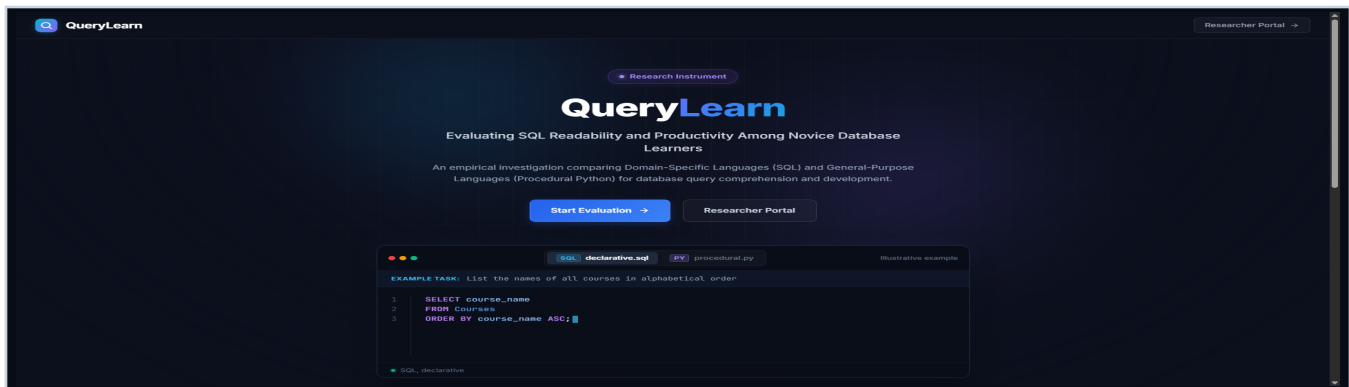
Actor Goals, Operational Boundaries, and Included Verification / Cloud Sync Dependencies



Formal Specification: Depicts 14 discrete use cases organized within the QueryLearn platform boundary. The Participant actor executes registration (UC-01), reference exploration (UC-02), readiness checks (UC-03), sandboxed query execution (UC-04), task submission (UC-05 with include dependencies for automated oracle grading and Supabase cloud sync), reading comprehension (UC-06), and post-condition feedback (UC-07). The Researcher actor authenticates (UC-09), monitors live participants (UC-10), reviews Pass vs. Fail telemetry (UC-11), inspects paired time ratios (UC-12), triggers scaling benchmarks (UC-13), and exports datasets (UC-14).

■ **What you say to your teacher / defense panel:** "Every use case in this diagram maps directly to our experimental protocol. Participant use cases collect our primary empirical telemetry (reading accuracy, task correctness, time-on-task, and cognitive ratings), while researcher use cases compute real-time medians and export clean data for statistical hypothesis testing."

Screen 1: Landing Page (The Welcome Screen)

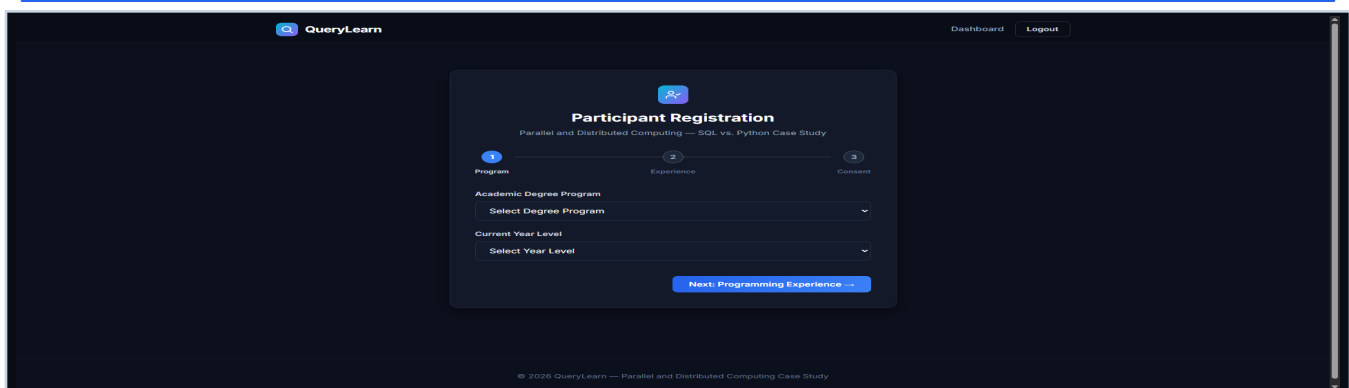


What this screen does: Welcomes participants, explains the study overview, and provides clear buttons for students to start the test or researchers to sign in to the dashboard.

Key elements: Large 'Start Evaluation' button for students, 'Researcher Login' for administrators, and 3 feature summary cards explaining the timed tasks and automatic grading.

■ ■ **What you say to your teacher / defense panel:** "This is the entrance portal. Students click 'Start Evaluation' to register anonymously, while researchers use the login to access real-time analytics and CSV exports."

Screen 2: Registration & Ethics Consent Wizard



What this screen does: Collects student background (degree program, year level [restricted to 2nd Year & Above], prior SQL and Python experience) and presents official ethics consent before assigning an anonymous, sequentially unique Study ID (P017+ via Supabase cloud).

Key elements: Degree program dropdown (BSCS, BSIT, BSIS), Year Level dropdown (2nd, 3rd, 4th Year; freshmen excluded), experience selectors, ethics agreement checkbox, and automatic assignment to a balanced testing sequence (Sequences 1 to 4).

■ ■ **What you say to your teacher / defense panel:** "We protect student privacy by generating an anonymous Study ID (like P017 or P018) via our shared Supabase cloud persistence layer. In accordance with the panel's directive, freshmen are excluded, focusing the sample strictly on 2nd to 4th year computing students across balanced Latin Square sequences."

Screen 3: Language Reference Guide (The Study Guide)

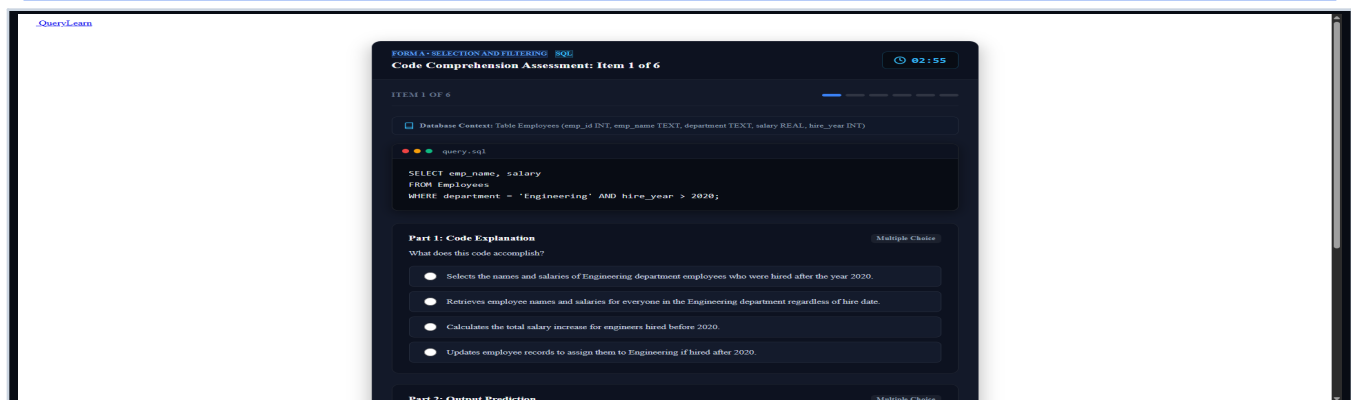


What this screen does: Teaches the 6 necessary coding patterns in both SQL and Python with plain-English summaries before the timed evaluation starts.

Key elements: Tab buttons to switch between SQL and Python, syntax-highlighted code blocks, and a one-sentence 'What it does' summary under every example.

■ **What you say to your teacher / defense panel:** "To ensure complete fairness, the guide uses an isolated university catalog schema (Courses and Departments) that never appears in actual tasks, teaching the rules without giving away any answers."

Screen 4: Code Comprehension Assessment (Reading Test)

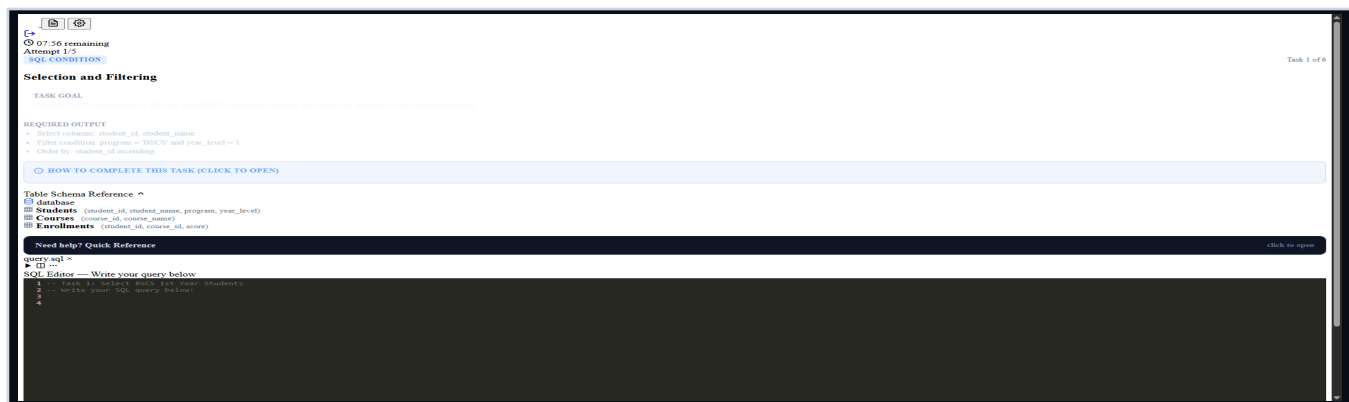


What this screen does: Tests reading comprehension before code writing. Students inspect 6 code snippets and answer: (1) what the code does (0-2 pts) and (2) what output it produces (0-1 pt).

Key elements: Read-only code window, multiple-choice explanation radio buttons, multiple-choice output prediction, 3-minute automatic countdown timer, and 18 points maximum score.

■ **What you say to your teacher / defense panel:** "Reading code is a distinct skill from writing code. We test comprehension first so we can measure mental readability independently of typing speed or syntax typos."

Screen 5: SQL Task Workspace (The Writing Test)

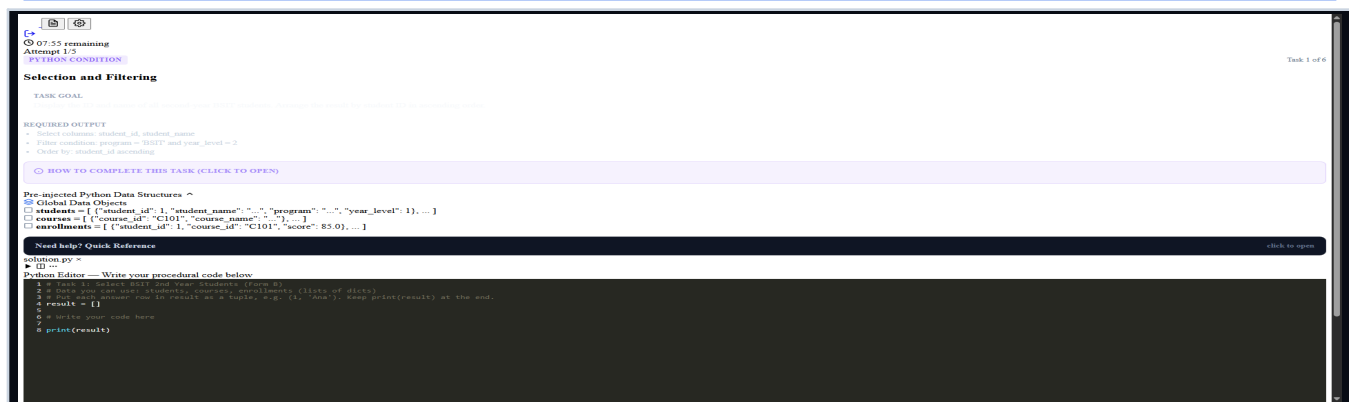


What this screen does: The live workspace where students write SQL queries for Tasks 1 to 6. Code runs against SQLite with instant output tables and automated test validation.

Key elements: 8-minute countdown timer, attempt counter (1/5), task requirements, collapsible schema tree, collapsible 'Need Help?' quick reference, CodeMirror SQL editor, Run button, and Submit button.

■ **What you say to your teacher / defense panel:** "The workspace provides real-time feedback. When students click Run, their query executes safely. When they click Submit, their code is validated against 3 hidden test databases with edge cases."

Screen 6: Procedural Python Task Workspace



What this screen does: The counterpart workspace for Python tasks. Pre-injects in-memory data structures (students, courses, enrollments) so students can write pure data manipulation logic.

Key elements: Identical 8-minute timer, attempt counter, data structure reference, Python CodeMirror editor, starter template code, and console output panel.

■ **What you say to your teacher / defense panel:** "The Python workspace uses the exact same layout, timer, and attempt limits as SQL. This ensures that the only variable being tested is the programming language itself."

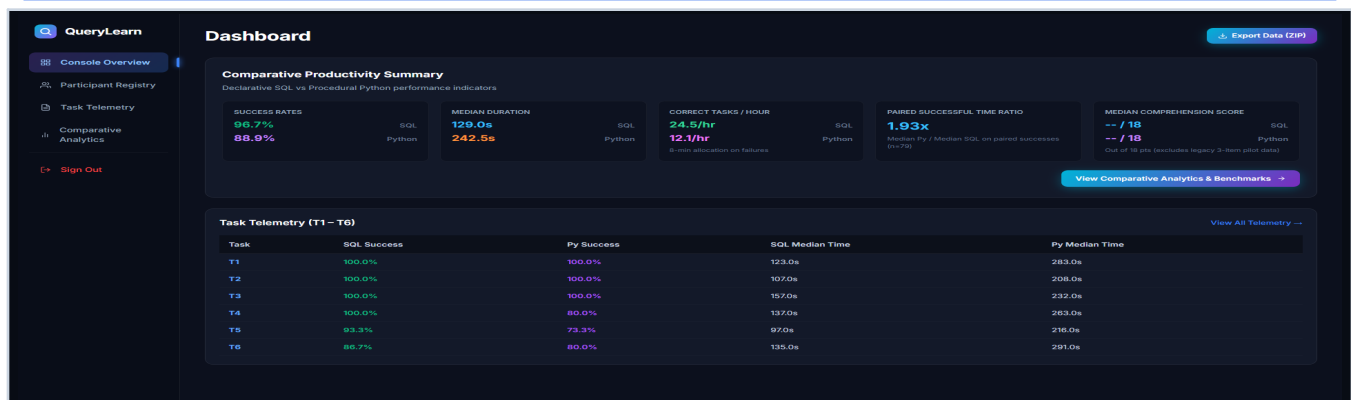
Screen 7: Post-Task Feedback Survey (Cognitive Ratings)

What this screen does: Measures student experience immediately after finishing each language: 5 agreement items on readability and debugging, mental effort, fatigue, and written comments.

Key elements: Symmetrical 5-column Likert scale (1=Strongly Disagree to 5=Strongly Agree), separated 'Not applicable' option, 1-5 mental effort rating, 1-5 fatigue rating, and qualitative textareas.

■ **What you say to your teacher / defense panel:** "We capture subjective cognitive load right after the tasks while it is fresh in the student's mind. 'Not applicable' is stored as NULL so it does not distort statistical numerical averages."

Screen 8: Researcher Analytics Dashboard



What this screen does: The secure administrative portal showing live telemetry, comparative statistics between SQL and Python, completion rates, explicit Right vs. Wrong (Pass/Fail) ratios, and raw CSV data downloads.

Key elements: Tile 1 with explicit Right vs. Wrong counts (e.g. 14 Right / 2 Wrong) and split percentages (87.5% Right, 12.5% Wrong), Task Telemetry Table (T1-T6), and one-click CSV export buttons.

■ **What you say to your teacher / defense panel:** "The dashboard gives researchers complete visibility. It calculates medians, tracks time-on-task, shows exact Right vs. Wrong counts and percentages, and exports raw data for statistical analysis in SPSS or R."

Oral Defense Cheat Sheet: Top Questions Your Evaluation Panel Will Ask

Study these 12 questions and speak the green answers clearly during your defense presentation:

1. 'Why did you build a web app instead of just giving students Google Forms or paper?'

■ ■ What you say to your teacher / defense panel: "Because a web app records exact millisecond typing time, tracks every failed attempt, executes code safely in a sandbox, and auto-grades solutions against edge cases without human bias."

2. 'How can students with no prior formal database course answer these tasks?'

■ ■ What you say to your teacher / defense panel: "We provided three beginner scaffolds: (1) a Reference Guide with plain-English summaries, (2) an untimed practice sandbox with a readiness check, and (3) a collapsible 'Need Help?' quick reference box inside the task window. All references use fake course data so they don't give away task answers."

3. 'Why are first-year freshmen excluded from the participant demographic?'

■ ■ What you say to your teacher / defense panel: "Per evaluation panel directive, freshmen have not yet completed foundational algorithmic programming courses. Restricting our evaluation population to 2nd, 3rd, and 4th-year students ensures participants have baseline programming literacy, allowing our experiment to measure the genuine readability differences of SQL without confounding complete programming novice barriers."

4. 'How does the platform handle multiple students registering on different laboratory computers?'

■ ■ What you say to your teacher / defense panel: "QueryLearn connects to a shared Supabase PostgreSQL cloud database. When a student registers on any computer or serverless instance, it atomically queries the shared database to assign the next sequentially unique Study ID (P017, P018, P019, etc.), and instantly synchronizes all task attempts and surveys so the teacher's dashboard displays aggregated live results across the entire laboratory in real-time."

5. 'What if a student just guesses or hardcodes the answer like `print([1, 2, 3])`?'

■ ■ What you say to your teacher / defense panel: "They cannot cheat! The system tests their code against hidden test datasets with edge cases (like tie scores, missing values, and empty rows). If their code does not genuinely calculate the right logic, it fails automatically."

6. 'What are the main metrics or numbers you are collecting?'

■ ■ What you say to your teacher / defense panel: "We collect four primary numbers: (1) Reading score (out of 18), (2) Task success rate (exact Right vs. Wrong counts and percentages), (3) Time spent solving each task (seconds), and (4) Number of attempts (from 1 to 5). We also collect self-reported mental effort and fatigue ratings."

7. 'Why do you test code comprehension before task writing?'

■ ■ What you say to your teacher / defense panel: "Because reading and writing are two different cognitive skills. Testing comprehension first lets us measure how easily a beginner understands the syntax before they have to worry about typing speed or syntax typos."

8. 'Why does each task have an 8-minute timer?'

■ ■ What you say to your teacher / defense panel: "To keep the experiment controlled and standardized. If there were no timer, one student might spend 40 minutes on one task, which ruins the time comparison and exhausts the participant."

9. 'What happens if the timer runs out before they finish?'

■ ■ What you say to your teacher / defense panel: "The app automatically saves whatever they did, marks the task as timed out, and smoothly moves them to the next task so they don't get stuck forever."

10. 'Why is Python longer than SQL for tasks 4, 5, and 6?'

■ ■ What you say to your teacher / defense panel: "Because SQL has built-in database keywords like JOIN and GROUP BY, while in Python you have to manually build dictionaries, compute sums, and check set memberships. That difference in difficulty is exactly what this study is measuring!"

11. 'How do you keep participant identities anonymous?'

■ ■ What you say to your teacher / defense panel: "Students never type their real name. The system assigns an anonymous Study ID (like P001, P002, P017). All research data and exports only use that ID, keeping participant privacy 100% protected."

12. 'What will you do with the final data collected in the dashboard?'

■ ■ **What you say to your teacher / defense panel:** "We export the data as clean CSV files and run standard statistical tests (like Wilcoxon paired tests for ratings and paired t-tests for task times) to prove whether SQL or Python was statistically significantly better."